

Project Cloak: A High-Performance, Privacy-First Agentic Memory Platform

Project Cloak is a new agentic memory platform designed for strictly local, CPU-native execution, with strong privacy and compliance guarantees. Cloak builds on the core ideas of AgentMem v2, treating memory as a *deterministic, structured* entity separate from learned control ¹. It maintains an entity-centric knowledge graph, semantic clustering indices, and an episodic trace log, while using a lightweight, policy-driven controller to govern retrieval, storage, and forgetting. Crucially, **every decision is fully auditable and reproducible** – cloaked agents log complete context traces so outcomes can be deterministically replayed (as emphasized in AgentMem’s tenets: “Auditability above convenience. Every decision must be reproducible from its trace” ²). LLMs and other generative tools are treated as non-authoritative utilities (used only for perception and support), and the system enforces a clear separation between **Perception (data ingest and canonicalization)**, **Decision (retrieval and reasoning)**, and **Explanation (trace logging and human feedback)**.

Design Principles

Project Cloak adheres to strict technical tenets, adapted from AgentMem v2, with additional emphasis on privacy and governance:

- **Local Execution & Privacy:** All computation (embedding, retrieval, clustering, reasoning) occurs on-device on CPU. No cloud or GPU-based services are used. Data never leaves the host, ensuring compliance with privacy policies.
- **Auditability & Deterministic Replay:** Every user interaction and agent decision is logged in an append-only trace (the **episodic store**). As AgentMem v2 mandates, “Every decision can be replayed from its context trace, including which memory IDs were retrieved and in what order” ³. The FSM-based execution loop and fixed policies guarantee that given the same inputs and memory state, the agent’s actions are deterministic.
- **Structured Memory (Deterministic Representation):** Memory is never an unstructured blob. Cloak’s core memory is an entity-relationship graph (e.g. Neo4j or SQLite-based) with timestamped, typed edges ⁴. Semantic clusters index related entities. **“Structure controls learning; learning controls memory”** – we do not rely on end-to-end LLM parameter updates for memory. Instead, **only explicit operations and feedback update the graph** ⁵.
- **Governed Memory Updates:** A dedicated **Memory Governance Module** intercepts all write operations. It enforces policies (e.g. privacy filters, schema constraints) before any new fact is committed. Human corrections are treated as “first-class memory” – i.e. permanent updates to the graph rather than ephemeral logs ⁶.
- **Learning Through Feedback:** The controller adapts its retrieval policy via reinforcement-style learning. Bandit and self-supervised loops adjust how much context is retrieved based on past outcomes. We additionally incorporate a **latent predictive learning** component inspired by JEPA (Joint Embedding Predictive Architecture) to refine memory representations (see *Learning Subsystem*).

- **Graceful Degradation:** All advanced components (clustering, learned policy) have rule-based fallbacks. The system functions correctly from day one, even with minimal training data.

These principles ensure Cloak is trustworthy and robust: each fact in memory carries provenance, each decision carries a trace, and every update is explicit and auditable.

High-Level Architecture

The overall system is organized into modular layers (Figure 1). Inputs enter through a **Perception/Canonicalization layer**, extracting entities and semantic triples from raw data (text, images, etc.), and generating embedding vectors. These feed into the **Structured Memory Layer**, composed of:

- **Entity Graph Store:** A deterministic knowledge graph (e.g. a local graph DB) holding entities (nodes) and timestamped relationships (edges) ⁴.
- **Vector Store:** A local similarity index (e.g. FAISS or CPU-optimized HNSW) for embedding search.
- **Semantic Clusters:** A clustering index (e.g. HDBSCAN) linking semantically related graph entities.
- **Episodic Store (Trace Log):** An append-only log of context traces and outcomes ⁷.

The **Memory Controller** (an FSM) governs the agent's loop. At query time it issues retrieval parameters (depth, breadth) to the **Retrieval Engine**, which performs multi-phase recall (vector search + graph traversal + cluster expansion). Retrieved memory chunks are passed to the **Reasoning Subsystem** (collections of utility agents or LLM-based modules) to produce actions or answers. All decisions, along with retrieved memory IDs and confidence, are logged to the episodic store with a unique TraceID.

Later, a background **Learning Loop** processes trace logs and feedback. It updates the controller's policy (e.g. UCB values), decays fact confidences, and invokes the **JEPA-based Latent Learner** to refine embeddings (see below). Human feedback received asynchronously is scored and fed back into the graph immediately (closing the loop) ⁶.

Figure 1 (Architecture): High-level component diagram of Project Cloak, showing input ingestion, memory layers (Graph, Vector, Clusters, Episodic log), the FSM-based controller, retrieval engine, reasoning agents, and learning loop. (Diagram source code in Appendix A.)

Execution Model (FSM)

Cloak's runtime behavior is implemented as an explicit **finite state machine (FSM)**. This deterministic control flow ensures reproducibility and simplifies auditing. Typical states include: **Idle**, **Ingest**, **Retrieve**, **Decide**, **StoreCheck**, **LogTrace**, and **Decay**. For example, on receiving a query the FSM transitions Idle→Evaluate→Retrieve (controller selects retrieval arm)→Decide (run reasoning)→StoreCheck (apply governance)→(Store/Defer)→LogTrace→Idle. A periodic Decay transition applies confidence decay and cleanup on schedule.

This FSM is analogous to AgentMem's controller state machine, but extended for Cloak's governance. As in AgentMem v2, **every transition and action is logged**, and no state is taken via an opaque LLM step. This means the system can deterministically replay: given a logged trace, one can reenact each state transition and memory access to verify the outcome.

State Diagram (Appendix B) details Cloak's execution states. Notably, LLM calls (for natural language parsing or answer generation) occur in well-defined states (e.g. within a "Reason" sub-state) and do not directly alter memory or control flow; they are treated as oracles whose output is separately logged.

Memory Subsystem

Cloak's **memory subsystem** stores and retrieves facts under strict governance:

- **Canonicalization & Ingestion:** Raw inputs (user text, sensor data, etc.) are first parsed by a deterministic pipeline (e.g. named-entity recognition, relation extraction) into structured triples (subject-predicate-object with confidence, timestamp) ⁸. Simultaneously, a lightweight embedding of the input is computed (e.g. sentence or chunk embedding).
- **Governed Update Pipeline:** Before committing new facts, the **Governance Module** validates them against policies (e.g. privacy filters, schema rules). Only sanctioned triples are written to the **Entity Graph**; each edge includes metadata (confidence, source, timestamp) ⁴. The new fact's embedding is linked as a source node or appended to the **Vector Store**, and the clustering index is updated to reflect its association with existing entity clusters. If confidence is low, a "defer" path skips storage (like AgentMem's defer gate ⁹).
- **Cluster Index:** Entities are semantically clustered using an on-device algorithm (e.g. incremental HDBSCAN). Clusters speed recall by grouping related entities: retrieval can expand along cluster neighbors for richer context. (AgentMem v2 also used "soft semantic clustering" to aid recall ¹⁰.)
- **Episodic Store:** Each query/decision is logged as a `ContextTrace` record ¹¹. This includes `trace_id`, input, retrieved memory IDs, reasoning steps, final decision, and outcome. This trace is append-only and immutable, enabling offline learning and audit. Importantly, **human corrections are treated as new facts** – when a user submits feedback, it is written directly into the graph (closing the feedback loop) rather than just tagged as metadata ⁶.
- **Retrieval Engine:** On query, Cloak performs a multi-phase recall. First, it vector-searches for top-k similar content (if applicable). Then it traverses the knowledge graph for entities related to the query terms up to a chosen depth. Finally, it expands through clusters to include nearby semantic content. The controller (via its chosen "arm") determines depth and breadth parameters. This hybrid recall ensures relevant context: for example, a rich history query yields more connected facts than a novel one ³.
- **Controlled Forgetting:** Long-lived facts are subject to confidence decay (TTL). As in AgentMem, confidences are multiplied by a decay factor over time, and stale edges (confidence below threshold) can be pruned. This ensures outdated information does not "influence decisions" indefinitely ¹².

Together, these components realize a **deterministic, structured memory**. AgentMem v2 noted that "when memory is an unstructured vector blob, there is no causal chain between what was retrieved and what was decided" ¹³. Cloak's design maintains that causal chain: every retrieved fact is an explicit node or triple in the graph, and its role in reasoning is recorded in the trace.

Reasoning and Decision Agents

The **Reasoning Subsystem** consists of specialized agents that use the retrieved memory to form decisions. The architecture enforces strict separation:

- **Planner Agent:** Decomposes queries into sub-tasks, invokes other agents, or triggers new memory searches.
- **Memory Specialist:** Manages interactions with the memory subsystem (e.g. formulating retrieval queries, editing the graph).
- **Researcher / Search Agent:** Accesses external tools if needed (within privacy constraints), such as local document search or on-device models.
- **Analyst / Synthesis Agent:** Performs final integration of information (e.g. calculating answers, summarizing).

These may be implemented with fixed code or LLMs, but critically **their outputs do not autonomously update memory**. All facts inserted back into memory go through the canonicalization/governance pipeline. This upholds the principle that **LLMs are non-authoritative utilities**: they can suggest facts, but only the pipeline can commit them. (This echoes AgentMem v2's tenet that "the knowledge graph is never mutated by gradient descent. Only heuristics and human feedback write to it" ⁵.)

Learning Subsystem (JEPA Latent Learning)

To improve retrieval and representation over time, Cloak employs a self-supervised latent learning mechanism inspired by **JEPA**. In addition to the reinforcement policy loop, we train a JEPA-like encoder/predictor on stored context. Specifically:

1. A **Context Encoder** (e.g. a lightweight neural network) maps recent context vectors or graph substructures into embeddings.
2. A **Target Encoder** (momentum average of the context encoder) computes embeddings of future or masked information in memory.
3. A **Predictor Network** tries to predict the target embeddings from the context embeddings.

The JEPA loss encourages the context encoder and predictor to capture **invariant features of the data**. As the JEPA literature explains, this "operates on latent representation space rather than reconstructing in the raw input domain, effectively filtering out high-frequency noise and extraneous details and enabling focus on the core structure and dynamics of the data" ¹⁴. In practice, this means Cloak's memory representations become more semantically robust: similar contexts map to similar latent codes, aiding retrieval and reducing hallucination. This training runs locally on CPU in the background, using stored embedding batches (it does not require GPU-scale models). The result is a **Latent Invariant Model** whose encodings are used both for clustering updates and to refine confidence estimates on graph edges (e.g. reinforcing edges that conform to learned invariants).

By integrating JEPA-inspired learning, Cloak develops an internal world-model that aligns with observed outcomes, without ever letting opaque gradients directly alter the graph structure. In essence, we combine AgentMem's bandit approach with JEPA's predictive world modeling to continuously improve context relevance.

Memory Governance and Compliance

A hallmark of Project Cloak is built-in **memory governance** for compliance-sensitive environments. Every memory write goes through a “governance funnel”:

- **Policy Filters:** Content is screened (e.g. for PII, prohibited categories) before ingestion. Blacklisted entities or correlations are either blocked or flagged.
- **Schema Enforcement:** All triples must conform to a strict ontology. Unexpected entity or predicate types cause the insert to be rejected or deferred.
- **Access Logs:** All reads and writes to the memory are logged. This ensures “auditability above convenience” – any access can be traced to the exact query or feedback that triggered it ².
- **Deterministic Replay:** Because the execution is an FSM and logs are complete, regulators or engineers can replay any decision path in full. This meets “compliance-grade auditability” requirements even in stringent domains (e.g. finance or health).

The governance layer also supports a formal **Memory Update Policy Language**. Administrators can write rules (e.g. “never store user’s private keys”, “age-restrict certain content”) which the module enforces at runtime. Combined with the append-only trace store, these measures make Cloak suitable for high-assurance applications.

LLM Utilization and Separation of Concerns

Project Cloak uses LLMs and other AI models strictly as tools, not authorities. For example, an on-device model might convert speech to text (Perception) or summarize retrieved facts (Reasoning), but it does not autonomously alter the structured memory or control logic. This reflects our design strategy of *perception vs. decision vs. explanation*:

- **Perception Layer:** Deterministic parsers (or small local models) turn raw inputs into data. LLMs here serve as heuristic helpers (e.g. to extract triples from text) but their output is vetted by the governance pipeline.
- **Decision Layer:** The core reasoning, including retrieval policy and decision rules, is deterministic code or simple learned policies (e.g. UCB bandit as in AgentMem) ¹⁵. If an LLM is used (e.g. for creative summarization), its suggestions are integrated into a structured response and logged.
- **Explanation Layer:** The final trace and outputs are human-readable explanations of what facts were used and how. These are generated by tracing through the FSM and memory paths, not by simply printing LLM latent states.

This separation ensures Cloak does not rely on LLM chain-of-thought; instead, any generated reasoning is backed by explicit retrieved facts. As AgentMem v2 argued, without a causal memory chain “there is no learning” and every decision is made in a vacuum ¹³. Cloak remedies that by keeping the chain visible.

Implementation Notes

- **CPU-Native Optimization:** Cloak is implemented in a performant stack of C++/Rust for core modules, and uses optimized CPU ML libraries (e.g. TVM, ONNX Runtime) for any required model inference. Embedding models are small and quantized. Vector search uses memory-mapped indexes.

- **Determinism:** All pseudo-randomness is seeded and logged. The FSM is single-threaded to avoid nondeterminism (or concurrency is carefully synchronized). Data types and operations are chosen for bitwise reproducibility.
- **Storage:** The knowledge graph can use a lightweight embedded DB (e.g. SQLite with extensions, or an in-memory graph engine) to avoid external dependencies. The vector/cluster indices also reside locally (e.g. flat files or embedded KV stores).
- **Interfaces:** There is no external API – interaction is via a local CLI or application link. The system can run headless for automated tasks, with all inputs and outputs logged.
- **Extensibility:** In line with AgentMem’s goal of replaceable components ¹⁶, Cloak’s modules (e.g. clustering algorithm, embedding model, LLM) can be swapped out. But core data flows and governance remain fixed to ensure auditability.

Conclusion

Project Cloak delivers a high-performance, agentic memory platform that is **privacy-first and fully auditable**. It inherits the rigor of AgentMem v2’s design – using a structured graph memory with a learned controller ¹⁷ – and augments it with local execution guarantees and a JEPA-inspired latent learning layer. With its FSM-based execution and governed memory pipeline, Cloak ensures that every decision is traceable, explainable, and compliant. The resulting system can self-improve from experience while respecting strict operational constraints, making it suitable for enterprise and regulated environments where trust and privacy are paramount.

Appendix: UML Diagrams

Figure 1: System Architecture

```

flowchart LR
    subgraph Inputs ["User / Environment"]
        U((User Input))
    end
    U --> Ingestor["Canonicalization / Ingestor"]
    Ingestor --> GraphDB["Knowledge Graph (Node/Edge DB)"]
    Ingestor --> VectorDB["Vector Store (Embeddings)"]
    GraphDB --> ClusterIdx["Cluster Index"]
    VectorDB --> Retriever["Retrieval Engine"]
    ClusterIdx --> Retriever
    MemoryCtrl["FSM Controller"] --> Retriever
    Retriever --> Reasoning["Reasoning Agents"]
    Reasoning --> Output["Decision / Action"]
    Output --> Episodic["Episodic Trace Log"]
    subgraph Learning ["Background Learning Loop"]
        Episodic --> Learner["JEPA Learner & Policy Update"]
        Learner --> MemoryCtrl
        Learner --> GraphDB
    end
    subgraph Feedback ["Human / Policy Feedback"]

```

```

FeedbackMgr["Feedback Manager"]
FeedbackMgr --> GraphDB
FeedbackMgr --> MemoryCtrl
Reasoning --> FeedbackMgr
end

```

Figure 2: Execution State Machine (FSM)

```

@startuml
[*] --> Idle
Idle --> Ingest : NewData / Query
Ingest --> Evaluate : Canonicalization
Evaluate --> Retrieve : Always
Retrieve --> Decide : Memory retrieved
Decide --> StoreCheck : Facts ready
StoreCheck --> Store : Gate pass
StoreCheck --> Defer : Gate fail
Store --> Log : commit fact
Defer --> Log : skip fact
Log --> Idle
Idle --> Decay : Timer
Decay --> Idle
@enduml

```

Figure 3: Query Handling Sequence

```

@startuml
actor User
participant Controller
participant Memory
participant Retrieval
participant Reasoning
participant Episodic

User -> Controller: Query arrives
Controller -> Retrieval: get_retrieval_params()
Retrieval --> Controller: {depth,breadth} (from FSM)
Controller -> Episodic: log_trace(trace_id, "pending")
Retrieval -> Memory: retrieve_context(params)
Memory --> Retrieval: context_items
Retrieval -> Reasoning: context_items
Reasoning -> User: Decision + trace_id
User -> Episodic: feedback(trace_id, score, correction)
Episodic -> Controller: update_policy(score)

```

```
Episodic -> Memory: apply_correction(correction)
@enduml
```

Figure 4: Memory Update Pipeline

```
@startuml
actor Sensor
participant Canonicalizer
participant Governance
participant GraphDB
participant ClusterIdx
Sensor -> Canonicalizer: raw input
Canonicalizer -> Governance: extracted triples + embeddings
Governance -> GraphDB: vetted triples
Governance -> ClusterIdx: embeddings update
GraphDB -> Episodic: log_insertions()
@enduml
```

(Each diagram's source is shown above. Rendered diagrams should be appended for illustration.)

1 2 3 4 5 6 7 8 9 10 11 12 13 15 16 17 README.pdf

file:///file_00000000df38720c9d98f63954b27109

14 Joint-Embedding Predictive Architecture (JEPA)

<https://www.emergentmind.com/topics/joint-embedding-predictive-architecture-jepa>